

TRABAJO FINAL DE LA UNIDAD 5

Módulos para Odoo de VigRacing

Yago Otero Martínez

Índice

Resumen.....	3
Mis clases.....	3
¿Para que sirve?.....	3
Cosas a mejorar.....	3
Estructura del proyecto y contenido de cada fichero.....	6
Contenido del manifest.....	6
Contenido de cada clase.....	8
Contenido de los permisos.....	8
Contenido, creación, modificación y restricciones de las vistas.....	8
Restricciones de los campos.....	10
Implementación de dos modelos.....	12
Creación de mas modelos.....	13

Índice de contenido

Figura 1: Captura de pantalla de mi menú.....	3
Figura 2: Captura de pantalla de la clase Carrera.....	4
Figura 3: Captura de pantalla de la clase piloto.....	5
Figura 4: Captura de pantalla de como se mira la lista de pilotos.....	5
Figura 5: Captura de pantalla del listado de piezas.....	5
Figura 6: Archivo manifest.py.....	6
Figura 7: Clase Piloto en models.py.....	7
Figura 8: Clase Piezas en models.py.....	7
Figura 9: Clase Circuito en models.py.....	8
Figura 10: XML de la vista del view circuitos.....	9
Figura 11: XML del view de pilotos y piezas.....	10
Figura 12: XML de la creación del menu padre que cuelga de la lista de aplicaciones de Odoo.....	12
Figura 13: XML de los menús de arriba.....	12
Figura 14: Contenido del security.....	13
Figura 15: Ejemplo del tercer modelo.....	13

Resumen

Mi módulo de VigRacing está enfocado en mis anteriores trabajos realizados para esta asignatura, claramente estuve pensando en ideas para poder acompañar a mi equipo de carreras. Me baría gustado hacer algo mas elaborado pero al final por cuestiones de tiempo no pudo ser pero igualmente estoy contento con el resultado obtenido.

Mi módulo consiste en 3 ventanas , 3 clases y un único módulo anclado a la lista de aplicaciones

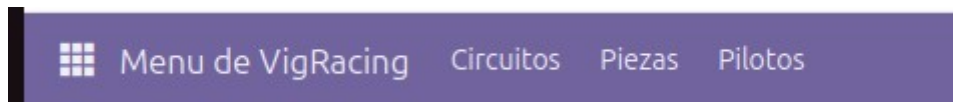


Figura 1: Captura de pantalla de mi menú

Mis clases

Las clases que utilicé para realizar el trabajo fueron:

- **Circuitos:** tendrá un título, la cantidad de kilometros, donde se encuentra haciendo una relación “many-to-one” con una tabla ya propia de Odoo pero además lo interesante de mi módulo una serie de pestañas como piezas utilizadas y pilotos que mas tarde explicaré cual es el fin de estas.
- **Piezas:** tendrán un nombre y la cantidad de stock que tiene el equipo en el almacén.
- **Pilotos:** nombre, piloto , edad , cantidad de mundiales ganados y cantidad de carreras ganadas (aquí me habría gustado hacer algo mas, una especie de historial con las últimas carreras y sus posiciones pero como dije por temas de tiempo fue inviable)

¿Para que sirve?

Mi módulo tiene el fin de contabilizar cuantas piezas fueron utilizadas a lo largo de cada carrera además de también llevar un recuento de quienes corrieron cada carrera porque aun que sí es común en la Formula 1 que los dos mismo pilotos corran casi siempre en los mismo monoplazas en otras categorías es muy común que se vayan intercambiando.

Entonces es por eso que hay check con cada registro dentro de cada circuito, así se puede llevar un conteo de cuantas piezas se utilizaron en cada.

Cosas a mejorar

A día de hoy solo puedo registrar tan solo una unidad de cada pieza , si yo ahora gastase 4 neumáticos no podría corregirlo correctamente , queda para alguna futura actualización

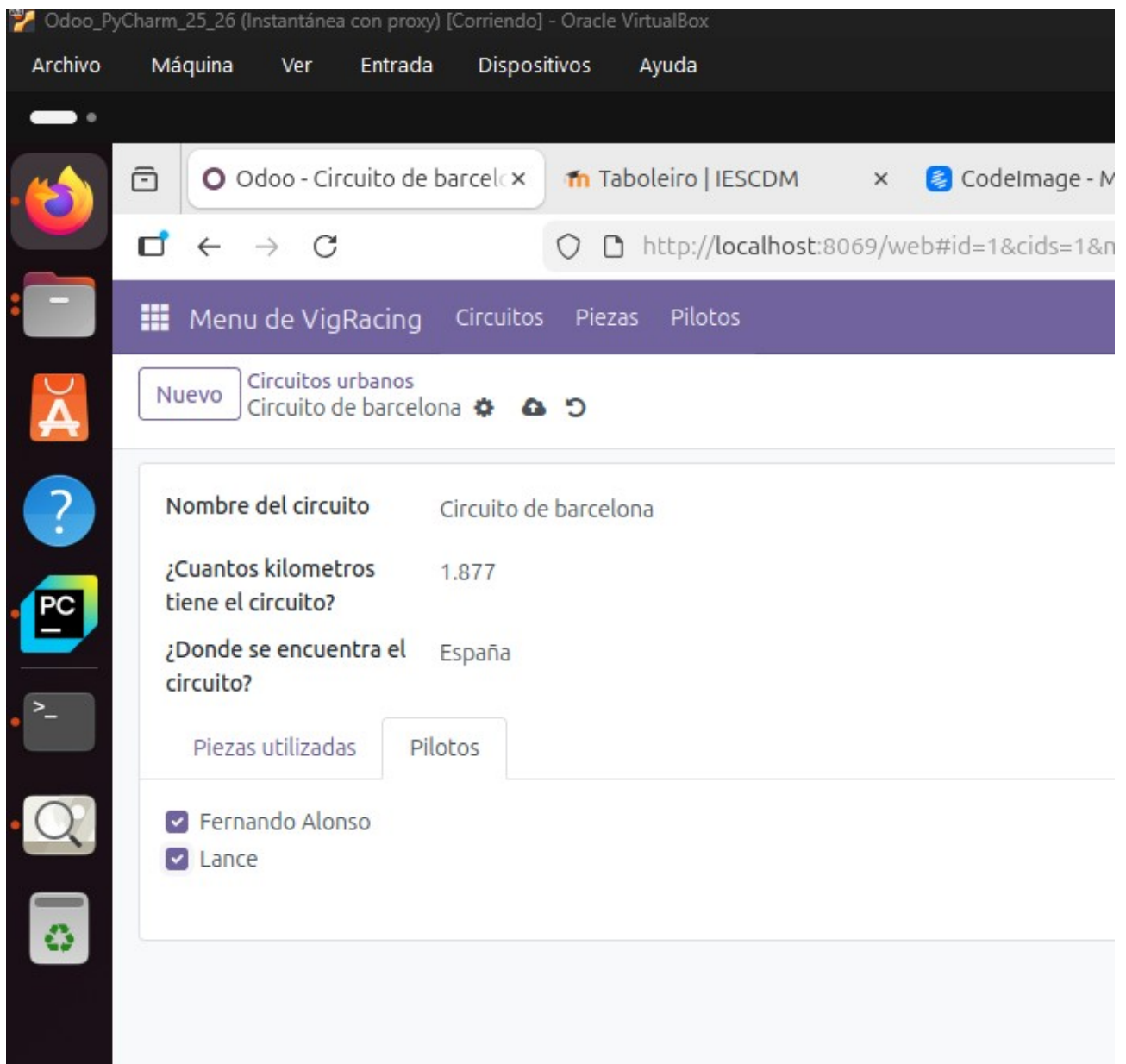


Figura 2: Captura de pantalla de la clase Carrera

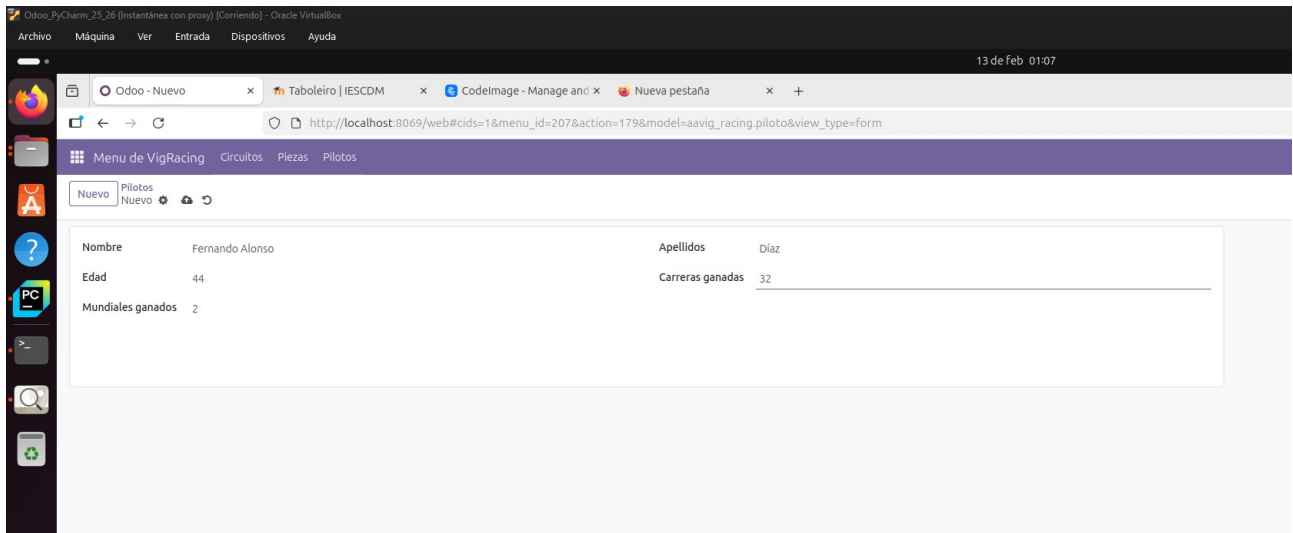


Figura 3: Captura de pantalla de la clase piloto

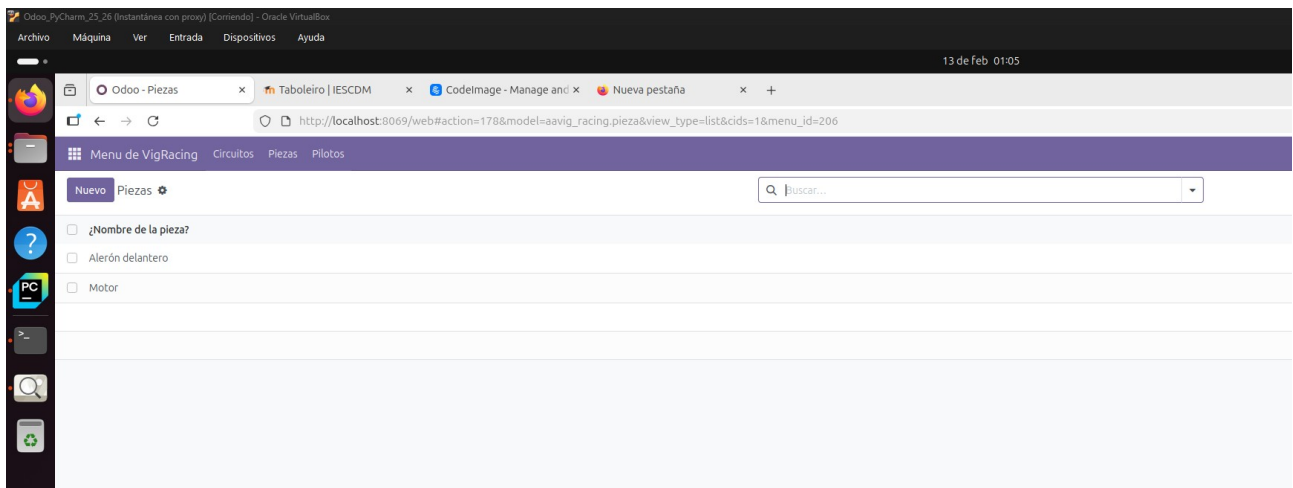


Figura 4: Captura de pantalla de como se mira la lista de pilotos

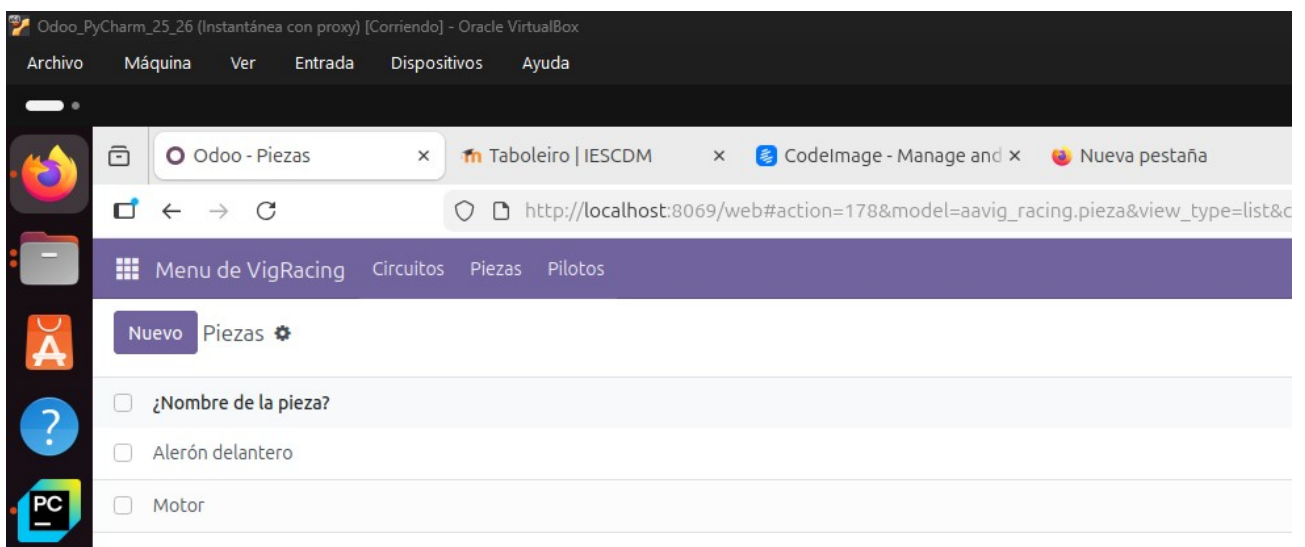


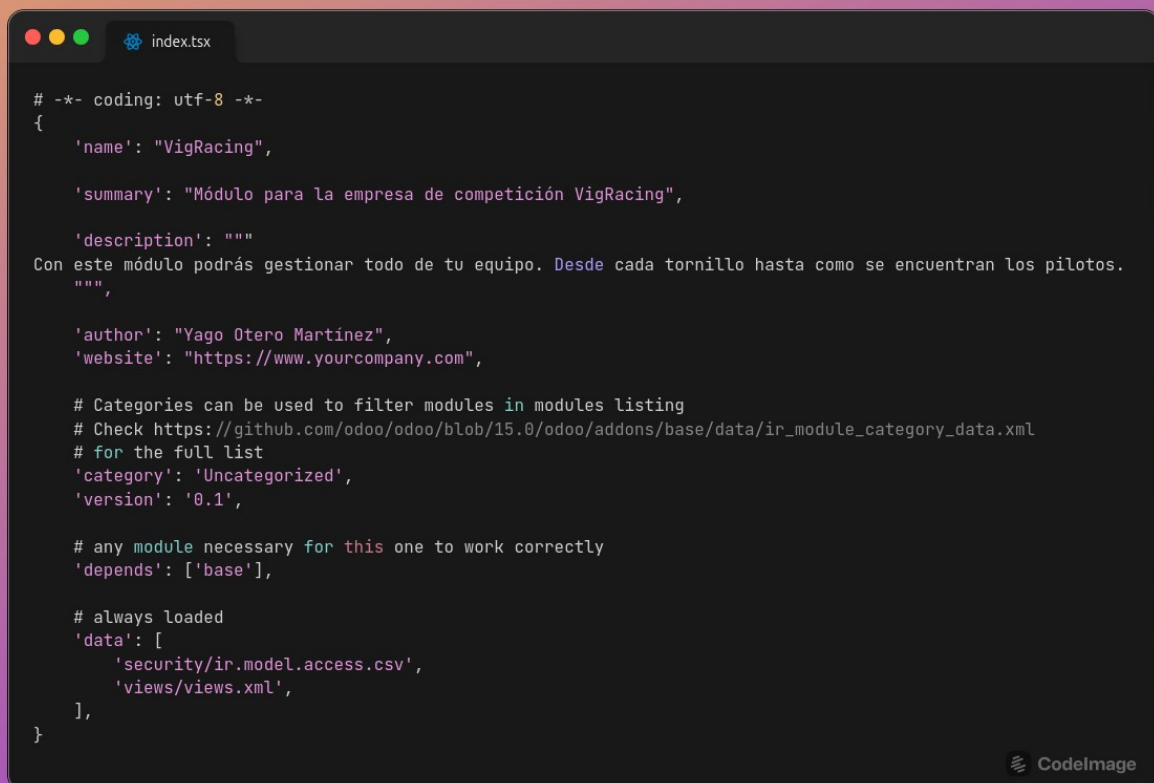
Figura 5: Captura de pantalla del listado de piezas

Estructura del proyecto y contenido de cada fichero.

Mi proyecto digamos que tiene varios paquetes, cada uno tiene una función específica pero es importante después iniciarlos en el “__init__.py”.

Contenido del manifest

De primeras hay un archivo que no está dentro de ninguna carpeta que es el “__manifest.py”. Este contiene toda la información sobre el módulo, su autor, resumen, de quien hereda clases ...



```
# -*- coding: utf-8 -*-
{
    'name': "VigRacing",

    'summary': "Módulo para la empresa de competición VigRacing",

    'description': """
Con este módulo podrás gestionar todo de tu equipo. Desde cada tornillo hasta como se encuentran los pilotos.
""",

    'author': "Yago Otero Martínez",
    'website': "https://www.yourcompany.com",

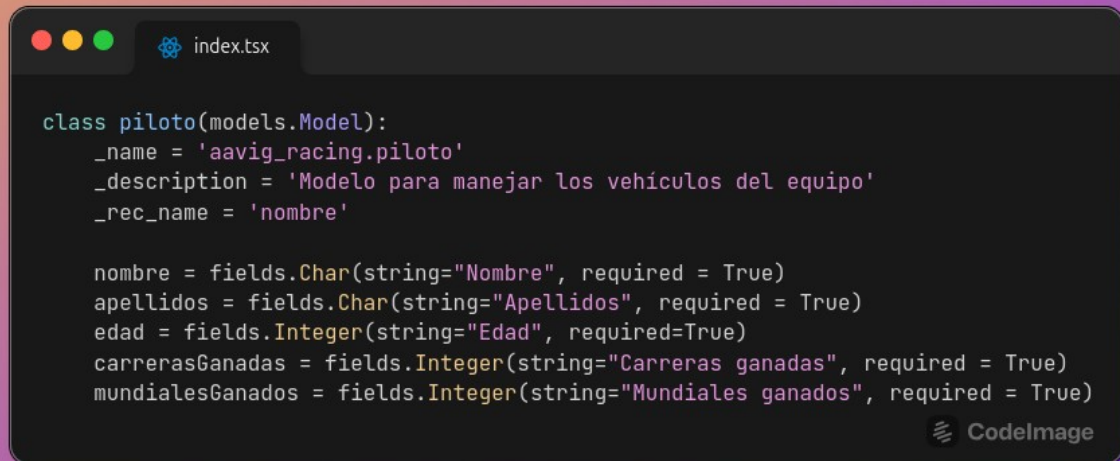
    # Categories can be used to filter modules in modules listing
    # Check https://github.com/odoo/odoo/blob/15.0/odoo/addons/base/data/ir_module_category_data.xml
    # for the full list
    'category': 'Uncategorized',
    'version': '0.1',

    # any module necessary for this one to work correctly
    'depends': ['base'],

    # always loaded
    'data': [
        'security/ir.model.access.csv',
        'views/views.xml',
    ],
}
```

Figura 6: Archivo manyfest.py

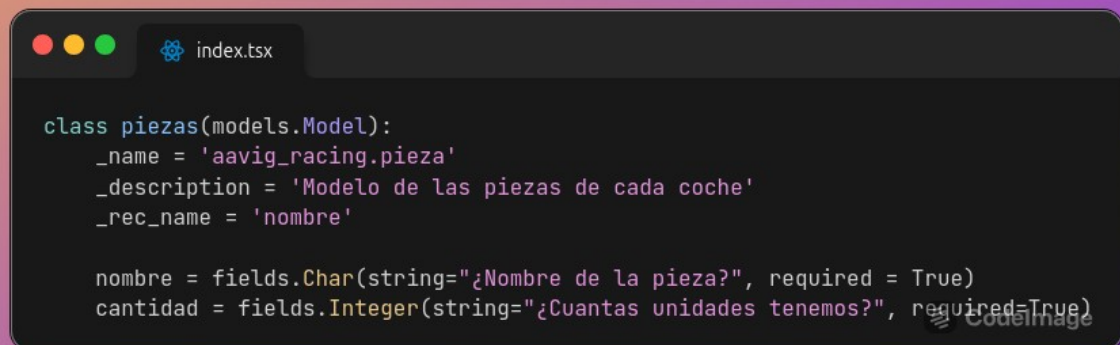
Después tenemos la carpeta “models”, donde tendremos otro __init__ pero este solamente va a iniciar las clases que utilicemos en nuestro proyecto. En mi caso, como eran solamente 3 clases decidí juntar todas en un mismo archivo pero para el futuro sería conveniente separarlas.



```
class piloto(models.Model):
    _name = 'aavig_racing.piloto'
    _description = 'Modelo para manejar los vehículos del equipo'
    _rec_name = 'nombre'

    nombre = fields.Char(string="Nombre", required = True)
    apellidos = fields.Char(string="Apellidos", required = True)
    edad = fields.Integer(string="Edad", required=True)
    carrerasGanadas = fields.Integer(string="Carreras ganadas", required = True)
    mundialesGanados = fields.Integer(string="Mundiales ganados", required = True)
```

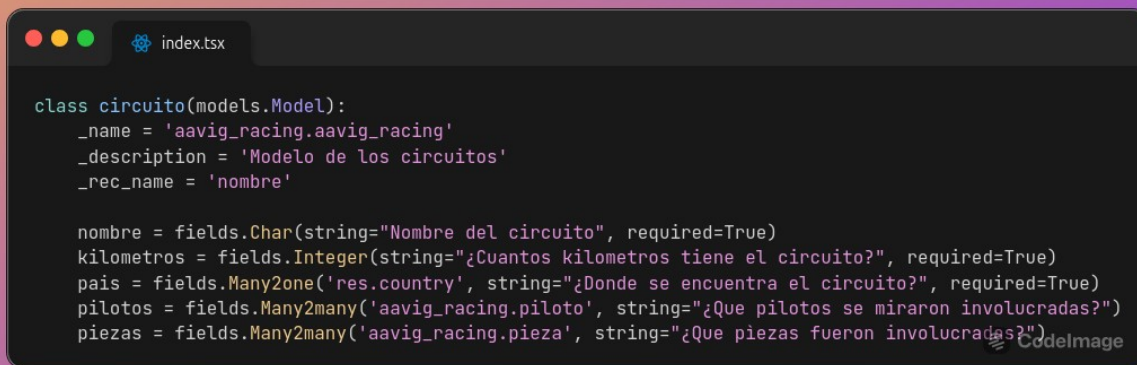
Figura 7: Clase Piloto en models.py



```
class piezas(models.Model):
    _name = 'aavig_racing.pieza'
    _description = 'Modelo de las piezas de cada coche'
    _rec_name = 'nombre'

    nombre = fields.Char(string="¿Nombre de la pieza?", required = True)
    cantidad = fields.Integer(string="¿Cuántas unidades tenemos?", required=True)
```

Figura 8: Clase Piezas en models.py

A screenshot of a code editor window with a dark theme. The window has a title bar with three colored circles (red, yellow, green) and a tab labeled 'index.tsx'. The code is written in Python and defines a class 'Circuito' that inherits from 'models.Model'. The class has three class attributes: '_name' set to 'aavig_racing.aavig_racing', '_description' set to 'Modelo de los circuitos', and '_rec_name' set to 'nombre'. It also has five fields: 'nombre' (Char, required), 'kilometros' (Integer, required), 'pais' (Many2one, required), 'pilotos' (Many2many), and 'piezas' (Many2many).

```
class circuito(models.Model):
    _name = 'aavig_racing.aavig_racing'
    _description = 'Modelo de los circuitos'
    _rec_name = 'nombre'

    nombre = fields.Char(string="Nombre del circuito", required=True)
    kilometros = fields.Integer(string="¿Cuántos kilometros tiene el circuito?", required=True)
    pais = fields.Many2one('res.country', string="¿Dónde se encuentra el circuito?", required=True)
    pilotos = fields.Many2many('aavig_racing.piloto', string="¿Que pilotos se miraron involucradas?")
    piezas = fields.Many2many('aavig_racing.pieza', string="¿Que piezas fueron involucradas?")
```

Figura 9: Clase Circuito en models.py

Contenido de cada clase

En las clases hay varias cosas que se repiten, “_name”, “_description” y “_rec_name” (este es como el toString() de Java).

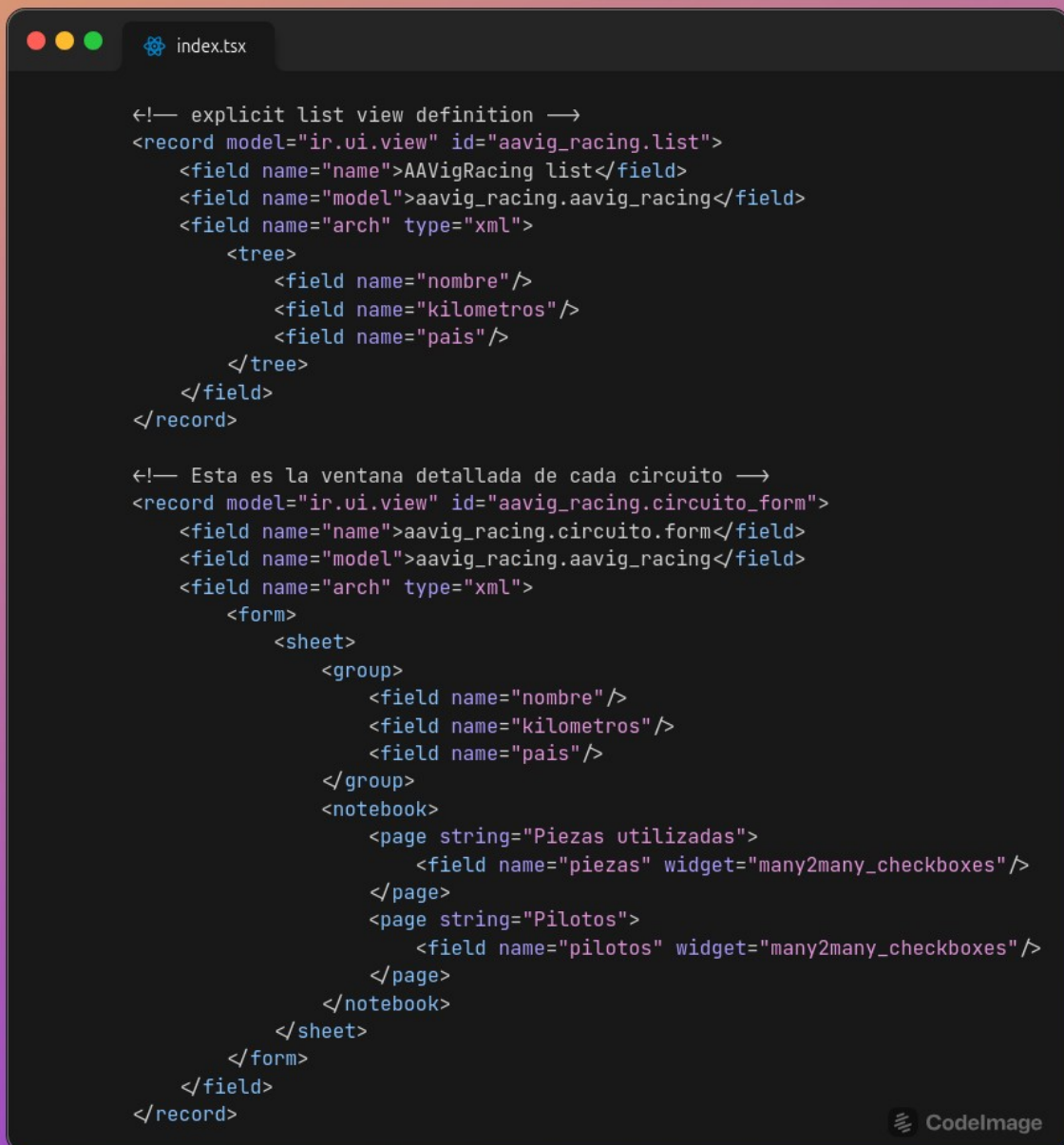
Más abajo están los parámetros que a ti te gustaría añadir a la clase, no hay mucho que explicar, pueden ser desde objetos hasta relaciones, como son las relaciones? Muy sencillas, One2One, One2Many y Many2One.

Contenido de los permisos

Hay un archivo muy importante en la carpeta security que es muy importante no saltar por alto, este marcará que permisos tendrán los usuarios sobre este, si lo pueden leer, modificar, alterar ... Nosotros siempre trabajamos dándole máximos permisos, entonces fue lo que hice, añadir cada clase al fichero y otorgarle el máximo de permisos

Contenido, creación, modificación y restricciones de las vistas

Esto va a ser un poco complicado de explicar, voy a dividir la vista en varias capturas de pantalla para explicarlo bien, primero esta la pantalla principal de circuito. Para ella tuve que hacerle una vista general y una vista mas personalizada para poder añadir los checkboxes de las relaciones

A screenshot of a code editor window titled 'index.tsx'. The editor has a dark background with light-colored text. The code is XML, defining two views. The first view is a list view for 'AAVigRacing' with fields 'nombre', 'kilometros', and 'pais'. The second view is a detailed form for 'AAVigRacing.circuito.form' with fields 'nombre', 'kilometros', 'pais', and two 'many2many_checkboxes' widgets for 'Piezas utilizadas' and 'Pilotos'. The code is color-coded: tags are in blue, attributes in green, and text values in pink. The editor has a standard macOS-style title bar with red, yellow, and green window control buttons. In the bottom right corner, there is a 'CodeImage' logo.

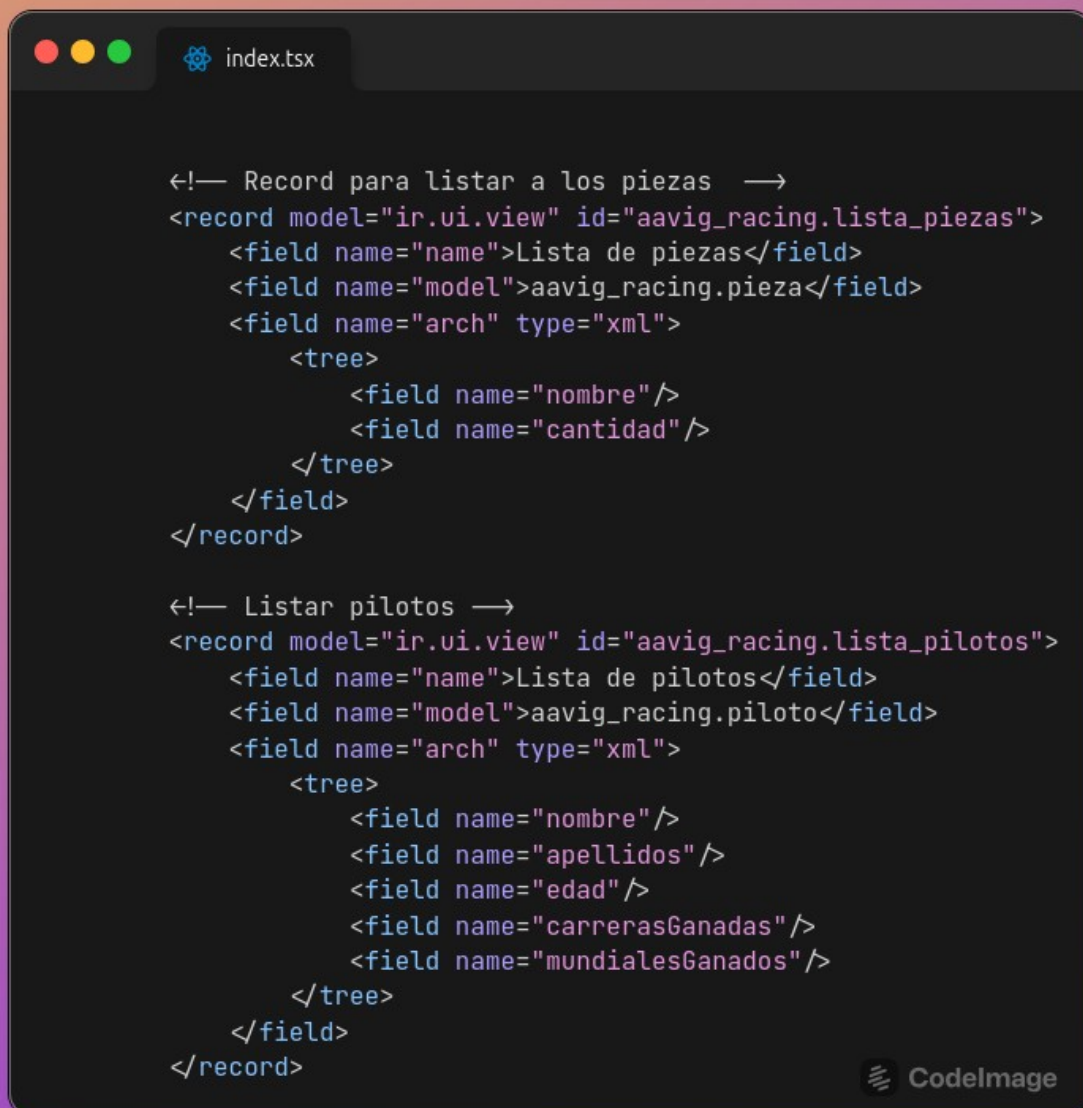
```
index.tsx

<!-- explicit list view definition -->
<record model="ir.ui.view" id="aavig_racing.list">
  <field name="name">AAVigRacing list</field>
  <field name="model">aavig_racing.aavig_racing</field>
  <field name="arch" type="xml">
    <tree>
      <field name="nombre"/>
      <field name="kilometros"/>
      <field name="pais"/>
    </tree>
  </field>
</record>

<!-- Esta es la ventana detallada de cada circuito -->
<record model="ir.ui.view" id="aavig_racing.circuito_form">
  <field name="name">aavig_racing.circuito.form</field>
  <field name="model">aavig_racing.aavig_racing</field>
  <field name="arch" type="xml">
    <form>
      <sheet>
        <group>
          <field name="nombre"/>
          <field name="kilometros"/>
          <field name="pais"/>
        </group>
        <notebook>
          <page string="Piezas utilizadas">
            <field name="piezas" widget="many2many_checkboxes"/>
          </page>
          <page string="Pilotos">
            <field name="pilotos" widget="many2many_checkboxes"/>
          </page>
        </notebook>
      </sheet>
    </form>
  </field>
</record>
```

Figura 10: XML de la vista del view circuitos

Después implementé las ventanas de Piezas y Pilotos, esta ya solo hice una vista mucho mas sencilla



```
index.tsx

<!-- Record para listar a los piezas -->
<record model="ir.ui.view" id="aavig_racing.lista_piezas">
  <field name="name">Lista de piezas</field>
  <field name="model">aavig_racing.pieza</field>
  <field name="arch" type="xml">
    <tree>
      <field name="nombre"/>
      <field name="cantidad"/>
    </tree>
  </field>
</record>

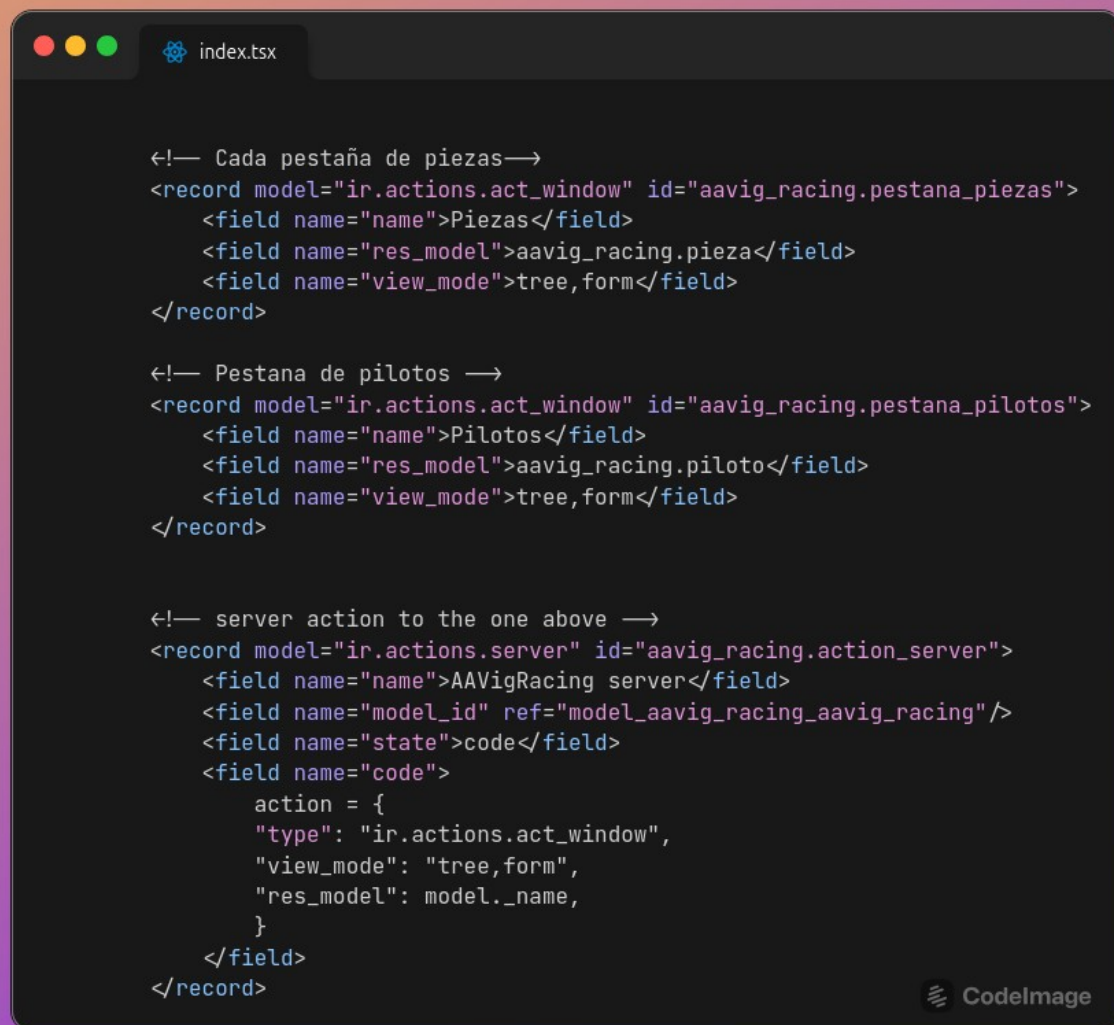
<!-- Listar pilotos -->
<record model="ir.ui.view" id="aavig_racing.lista_pilotos">
  <field name="name">Lista de pilotos</field>
  <field name="model">aavig_racing.piloto</field>
  <field name="arch" type="xml">
    <tree>
      <field name="nombre"/>
      <field name="apellidos"/>
      <field name="edad"/>
      <field name="carrerasGanadas"/>
      <field name="mundialesGanados"/>
    </tree>
  </field>
</record>
```

Figura 11: XML del view de pilotos y piezas

Restricciones de los campos

Es importante recalcar que cada vista debe de ir asociada a un model , tendrá un id unico de donde podrán colgar los menus y debe de tener asociado también a que clase va dirigida a parte del name que será como el título de la ventana.

Ahora que ya mencioné como funcionan los ids de arriba es importante decir como trabajan los submenus de arriba , para que cuando clickemos en Piezas , Carreras ... vaya a donde queramos eso primero lo tendremos que asociar como se muestra en la siguiente imagen

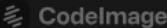


```
index.tsx

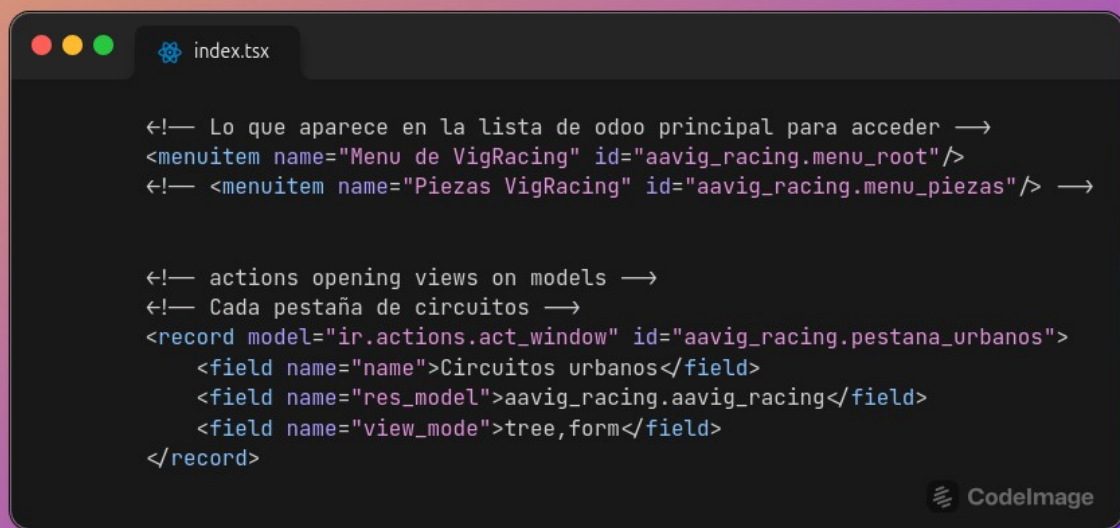
<!-- Cada pestaña de piezas -->
<record model="ir.actions.act_window" id="aavig_racing.pestana_piezas">
  <field name="name">Piezas</field>
  <field name="res_model">aavig_racing.pieza</field>
  <field name="view_mode">tree,form</field>
</record>

<!-- Pestaña de pilotos -->
<record model="ir.actions.act_window" id="aavig_racing.pestana_pilotos">
  <field name="name">Pilotos</field>
  <field name="res_model">aavig_racing.piloto</field>
  <field name="view_mode">tree,form</field>
</record>

<!-- server action to the one above -->
<record model="ir.actions.server" id="aavig_racing.action_server">
  <field name="name">AAVigRacing server</field>
  <field name="model_id" ref="model_aavig_racing_aavig_racing"/>
  <field name="state">code</field>
  <field name="code">
    action = {
      "type": "ir.actions.act_window",
      "view_mode": "tree,form",
      "res_model": model._name,
    }
  </field>
</record>
```



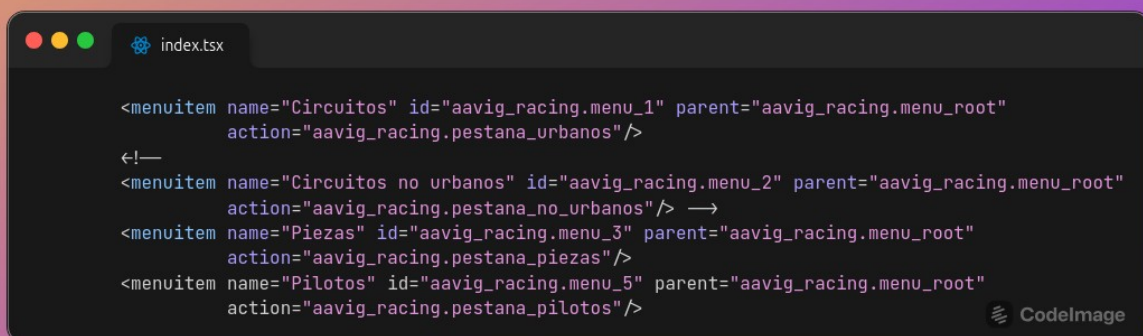
Y ahora si, después de asignar esos identificadores si podremos crear los menus

A screenshot of a code editor window titled 'index.tsx'. The code is XML for Odoo's menu system. It starts with a comment '←!— Lo que aparece en la lista de odoo principal para acceder →' followed by a menu item: '<menuitem name="Menu de VigRacing" id="aavig_racing.menu_root"/>'. Then another comment '←!— <menuitem name="Piezas VigRacing" id="aavig_racing.menu_piezas"/> →'. A second comment '←!— actions opening views on models →' is followed by '←!— Cada pestaña de circuitos →'. Then a record is defined: '<record model="ir.actions.act_window" id="aavig_racing.pestana_urbanos">'. Inside the record, there are three fields: '<field name="name">Circuitos urbanos</field>', '<field name="res_model">aavig_racing.aavig_racing</field>', and '<field name="view_mode">tree,form</field>'. The record is closed with '</record>'. The CodelImage logo is in the bottom right corner.

```
←!— Lo que aparece en la lista de odoo principal para acceder →
<menuitem name="Menu de VigRacing" id="aavig_racing.menu_root"/>
←!— <menuitem name="Piezas VigRacing" id="aavig_racing.menu_piezas"/> →

←!— actions opening views on models →
←!— Cada pestaña de circuitos →
<record model="ir.actions.act_window" id="aavig_racing.pestana_urbanos">
  <field name="name">Circuitos urbanos</field>
  <field name="res_model">aavig_racing.aavig_racing</field>
  <field name="view_mode">tree,form</field>
</record>
```

Figura 12: XML de la creación del menu padre que cuelga de la lista de aplicaciones de Odoo

A screenshot of a code editor window titled 'index.tsx'. The code defines four child menu items. The first is '<menuitem name="Circuitos" id="aavig_racing.menu_1" parent="aavig_racing.menu_root" action="aavig_racing.pestana_urbanos"/>'. The second is a comment '←!—' followed by '<menuitem name="Circuitos no urbanos" id="aavig_racing.menu_2" parent="aavig_racing.menu_root" action="aavig_racing.pestana_no_urbanos"/> →'. The third is '<menuitem name="Piezas" id="aavig_racing.menu_3" parent="aavig_racing.menu_root" action="aavig_racing.pestana_piezas"/>'. The fourth is '<menuitem name="Pilotos" id="aavig_racing.menu_5" parent="aavig_racing.menu_root" action="aavig_racing.pestana_pilotos"/>'. The CodelImage logo is in the bottom right corner.

```
<menuitem name="Circuitos" id="aavig_racing.menu_1" parent="aavig_racing.menu_root"
  action="aavig_racing.pestana_urbanos"/>
←!—
<menuitem name="Circuitos no urbanos" id="aavig_racing.menu_2" parent="aavig_racing.menu_root"
  action="aavig_racing.pestana_no_urbanos"/> →
<menuitem name="Piezas" id="aavig_racing.menu_3" parent="aavig_racing.menu_root"
  action="aavig_racing.pestana_piezas"/>
<menuitem name="Pilotos" id="aavig_racing.menu_5" parent="aavig_racing.menu_root"
  action="aavig_racing.pestana_pilotos"/>
```

Figura 13: XML de los menús de arriba

Implementación de dos modelos.

En mi caso no todos los modelos se relacionan entre sí sino que al final solo se relacionan conjuntamente en la clase Circuito , debido a que una carrera puede tener muchas piezas y muchos pilotos (Como se puede observar en la figura 9)

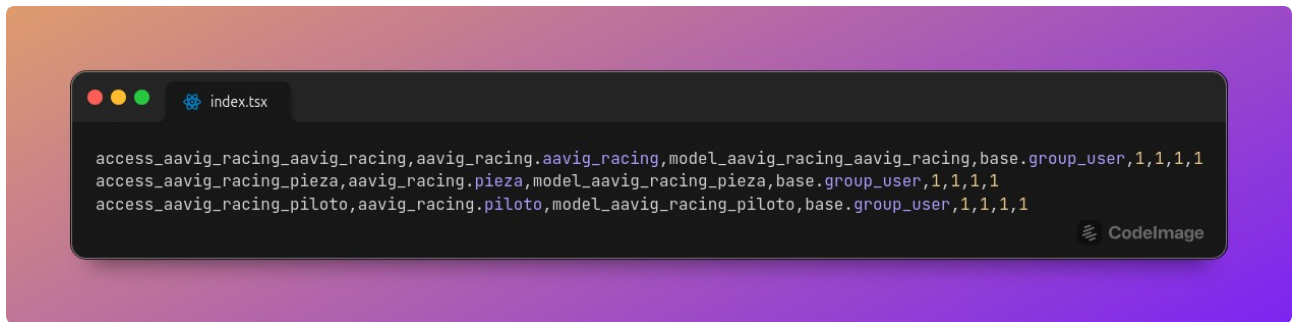


Figura 14: Contenido del security

Creación de mas modelos

Como el mínimo de modelos obligatorios fueron dos, yo decidí crear un tercero , no importa si es piezas o pilotos pero el caso es que al final le dan mas funcionalidad a mi módulo con lo que a su vez es mas interesante.

Nombre del circuito	Circuito de barcelona
¿Cuántos kilometros tiene el circuito?	1.877
¿Donde se encuentra el circuito?	España
Piezas utilizadas	Pilotos
☒ Alerón delantero	
☐ Motor	

Figura 15: Ejemplo del tercer modelo